

Assignment 6 – Device Driver

Description:

For my Assignment 6 device driver, I developed a character device accessible and provides Caesar-cipher encryption and decryption services through a simple file-I/O. When loaded, the module acquires an unused major number, creates a sysfs class, and instantiates the `/dev/main` node so that any user-space program can open, read, write, and issue `ioctl` commands against it. Internally, it keeps a fixed-size buffer to store the most recent string written and exposes two runtime-configurable parameters, a shift key, and an encrypt/decrypt mode that users can change on the fly via custom `ioctl` calls. Every write operation transforms alphabetic characters in place by shifting them forward or backward according to the current settings, and a matching read returns exactly the processed data. By using the standard `copy_from_user` and `copy_to_user` routines, the driver ensures safe transfer of data between user and kernel space, while its open and release handlers log device usage. Unloading the module cleanly reverses the setup—removing the `cdev` structure, deleting the `/dev/main` node and its sysfs entries, and freeing the major number—so that no kernel resources remain allocated. This design delivers a fully self-contained, dynamically configurable encryption service that integrates seamlessly with both scripts and interactive applications.

Approach:

First step was understanding the assignment and deciding on what I wanted to do. I started by carefully reading the entire assignment prompt from top to bottom, making note of every important requirement such so I wouldn't miss anything. I noted the need for a loadable kernel module in C, more specifically with open, read, write, `ioctl`, and release handlers as needed. I also saw that the grader would look for non-trivial functionality, so I knew I needed something slightly more than a "hello world" or super basic encryption, so I tried to find a more professional version. While brainstorming, I recalled basic cryptography from an online tutorial and settled on a Caesar cipher. I did a lot of research and found it's a old technique where each letter is shifted by a fixed offset in the alphabet. I Googled "Caesar cipher C implementation" to refresh my memory on the modulo arithmetic and boundary wrapping. That quick research proved to me it was simple enough to code in the kernel and complex enough to meet the assignment's criteria.

Next, I set up my development environment on the VM exactly as the instructions specified. I also made sure to make a copy of my VM so that if I broke anything I wouldn't have to go back to week one and re-do all those steps. I ran `sudo apt update`, installed `gcc-12`, and added `pahole` so I could inspect BTF data. And linking `/sys/kernel/btf/vmlinux` into the modules build directory. I ran `pahole --version` to confirm I had the correct version. With the environment all good, I cloned the assignment and noticed the directory structure a `Module/` folder for the kernel code and a `Test/` folder for the user-space program. I inspected the instructor's Makefiles in each folder without modification, then adjusted only the `BASENAME`, `FIRSTNAME`, and `LASTNAME` variables so my filenames matched exactly what the build system expected. Also I

created the main c files inside of test and module. Running make in each folder produced the .ko and the test executable on the first try, which was a good sign to move on.

Before writing any real logic, I put out the kernel module's skeleton. I added all the necessary includes, `<linux/module.h>`, `<linux/fs.h>`, `<linux/cdev.h>`, `<linux/uaccess.h>`, `<linux/ioctl.h>`, and `<linux/device.h>`, and declared the global variables for major number, device class, device struct, cdev, and the internal buffer. I defined a constant for ioctl namespacing and two ioctl commands for setting the shift key and toggling encrypt/decrypt mode. I wrote empty or starter functions with no functionality yet for the main logic I would need. `main_open`, `main_read`, `main_write`, `main_ioctl`, and `main_release`, and then registered them in a `file_operations` struct. I implemented `main_init` and `main_exit` to handle module loading and unloading, with careful error paths that undo previous steps if anything fails.

With the structure verified, I tackled the actual Caesar cipher logic in `main_write`. I copied data from user space with `copy_from_user`, capped the length to avoid buffer overflow, and null-terminated the buffer for safety. I then looped through every character. If it was in A–Z or a–z, I shifted it by the current key, wrapping around using modulo 26 arithmetic. Anything else passed through unchanged. I returned the number of bytes consumed so that `write()` behaved like a normal file call. After that, I wrote `main_read`, which checks the file offset, copies data back to user space via `copy_to_user`, advances the offset, and returns the byte count or zero for end-of-file. Next I worked on `main_ioctl`. I used `copy_from_user` to read an integer mode or key value from user space, then updated the global variables accordingly. I handled unknown commands by returning `-EINVAL`. This meant I could change the encryption behavior on the fly without unloading the module. I tested ioctl functionality quickly with a small C snippet that opened the device, sent a key and mode, and verified that subsequent writes produced the expected shifted output. Once the kernel side was solid, I moved on to the user-space tester in `Test/bayercalvert_aidan_HW6_main.c`. I included `<stdio.h>`, `<stdlib.h>`, `<fcntl.h>`, `<unistd.h>`, `<sys/ioctl.h>`, and `<string.h>`. I parsed command-line arguments into mode, key, and input, then opened `/dev/main` for read/write access. I issued two ioctl calls to set the key and mode, wrote the input string, read back the transformed data, null-terminated the result, and printed it. I handled errors at every step with `perror` and `exit`, so any failure I was able to see where stuff went wrong.

Finally, I ran the full test sequence. I unloaded any existing module, inserted the new .ko with `insmod`, and checked `dmesg` for the major number (511). I created `/dev/main` with `mknod` and `chmod 666` so my tester could open it without root. Then I ran `./bayercalvert_aidan_HW6_main e 5 "Hello, World!"` and saw `Mjqqt, Btwqi!`. I ran the decrypt test and got back `Hello, World!`. I also tested edge cases: empty strings, long inputs capped at 1023 bytes, and strings with no alphabetic characters. Everything behaved predictably. Along the way I fixed a typo in a `printk` message, removed an unnecessary `class_unregister`, and ensured `MODULE_LICENSE("GPL")` was present so the kernel didn't have any issues, which I read online to include this. This step-by-step process took to a fully functional program basic device driver program.

Issues and Resolutions:

The first issue I encountered was while testing the driver, I noticed that encrypting letters near the end of the alphabet produced unexpected characters (for example, shifting 'Z' by 3 yielded 'C' properly, but shifting 'Y' by 5 gave '^' instead of 'D'). I suspected an arithmetic error in my modulo logic. To isolate the bug, I added temporary `printf()` statements inside the write loop, logging both the original character and its numeric transform. Reading through the combined ASCII values and modulo results, I realized I had forgotten to add 26 before applying modulo when decrypting, which allowed negative values to slip through. I updated the formula to $((ch - base - key + 26) \% 26) + base$ for both uppercase and lowercase branches. After recompiling and retesting with boundary characters ('X', 'Y', 'Z', 'x', 'y', 'z'), the cipher consistently wrapped correctly. Removing the debug prints and committing the corrected modulo expression resolved all off-by-one errors.

The next issue I ran into was early on, every `ioctl` call returned `-EINVAL`, even though my user-space code seemed to pass the right command numbers. I remembered that the key byte must match exactly on both sides, so I printed the numeric values of my `_IOW(KEY, ...)` definitions in the module's init routine. Those values didn't line up with what the user program was sending. I discovered I had accidentally defined `KEY` as 'B' in the test code but used 'A' in the driver. To debug, I compared the output of `grep -R IOCTL_SET_KEY` in both source files and spotted the mismatch. After standardizing on `#define KEY 'A'` in both files, recompiling, and reloading, `ioctl(fd, IOCTL_SET_KEY, &key)` began returning zero as expected. This taught me to always verify that low-level constants align exactly across kernel and user-space boundaries.

Another issue I ran into was when running the test program, I sometimes got "Failed to open device: No such file or directory." I rechecked that the module was loaded, but `/dev/main` didn't appear automatically because `udev` hadn't picked up the new class. Using `ls /sys/class/main` confirmed the class existed but no device symlink. To fix this, I manually ran `sudo mknod /dev/main c <major> 0` and `chmod 666 /dev/main`. I then added a comment in my writeup reminding myself to always create the node before testing.

Screen shot of compilation:

Test compiling:

```
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Test$ make clean
rm *.o bayercalvert_aidan_HW6_main
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Test$ make
gcc -c -o bayercalvert_aidan_HW6_main.o bayercalvert_aidan_HW6_main.c -g -I.
gcc -o bayercalvert_aidan_HW6_main bayercalvert_aidan_HW6_main.o -g -I. -l pthread
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Test$
```

Module compiling:

```
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ make clean
make -C /lib/modules/`uname -r`/build M=/home/student/Desktop/csc415-assignment-6-device-driver-abccodes/Module clean
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-58-generic'
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-58-generic'
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ make
make -C /lib/modules/`uname -r`/build M=/home/student/Desktop/csc415-assignment-6-device-driver-abccodes/Module modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-58-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: aarch64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0
You are using: gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0
CC [M] /home/student/Desktop/csc415-assignment-6-device-driver-abccodes/Module/bayercalvert_aidan_HW6_main.o
MODPOST /home/student/Desktop/csc415-assignment-6-device-driver-abccodes/Module/Module.symvers
CC [M] /home/student/Desktop/csc415-assignment-6-device-driver-abccodes/Module/bayercalvert_aidan_HW6_main.mod.o
LD [M] /home/student/Desktop/csc415-assignment-6-device-driver-abccodes/Module/bayercalvert_aidan_HW6_main.ko
BTF [M] /home/student/Desktop/csc415-assignment-6-device-driver-abccodes/Module/bayercalvert_aidan_HW6_main.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-58-generic'
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$
```

Screen shot(s) of the execution of the program and how to build kernel module and test program:

Step 1:

First, compile both the kernel module and the user-space test application exactly as shown in the “Screen shot of compilation” section. In the Module/ directory you run make, which invokes the kernel build system to produce bayercalvert_aidan_HW6_main.ko, simultaneously, in the Test/ directory you run make again to build the bayercalvert_aidan_HW6_main executable. This is so both components are up to date and are compiled.

Step 2:

(Not needed in first download) Next, remove any existing instance of driver with `sudo rmmod bayercalvert_aidan_HW6_main` (to avoid “module already loaded” errors), then (this is needed) insert the freshly built module via `sudo insmod bayercalvert_aidan_HW6_main.ko`. Immediately afterwards, use `sudo dmesg | tail -n 5` to confirm successful registration and note the assigned major number (e.g., 511). Finally, create the character device node with `sudo mknod /dev/main c <major> 0` and `sudo chmod 666` (arbitrary high value for access) `/dev/main` so that any user can open and interact with it

```
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ sudo insmod bayercalvert_aidan_HW6_main.ko
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ sudo dmesg | tail -n5
```

```
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ sudo dmesg | tail -n5
[95305.934817] helloenc: Received 13 bytes from user, mode=1, key=5
[95305.934820] helloenc: Sent 13 bytes to user
[95305.934978] helloenc: Device successfully closed
[101709.997411] input: VMware DnD UInput pointer as /devices/virtual/input/input9
[102234.086269] helloenc: Module unloaded
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$
```

```
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ sudo mknod /dev/main c 511 0
mknod: /dev/main: File exists
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ sudo chmod 666 /dev/main
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$
```

Step 3:

With /dev/main in place, switch back to the Test/ directory and invoke the test application with the encrypt or decrypt arguments. For example:

Encrypt:

```
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Module$ cd ../Test/
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Test$ ./bayercalvert_aid
an_HW6_main e 5 "Hello, World!"
Result: Mjqqt, Btwqi!
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Test$
```

Decrypt:

```
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Test$ ./bayercalvert_aid
an_HW6_main d 5 "Mjqqt, Btwqi!"
Result: Hello, World!
student@student:~/Desktop/csc415-assignment-6-device-driver-abccodes/Test$
```